

**YENEPOYA INSTITUTE OF ARTS, SCIENCE,
COMMERCE AND MANAGEMENT**

Low Level Design (LLD)

Personalized Recommendations Engine for Smart City

Student Name: Mohammed Shahid

Register Number: 23BSAMD015

Course: BSc Artificial Intelligence, Machine Learning and Data Science

Industry Mentor: Mr. Sumit Kumar Shukla

Guided By: Ms. Fathimath Raheema U

Table of Contents

1. Introduction
2. Detailed Module Design
3. Algorithm Design
4. Data Structures
5. Function-Level Design
6. Error Handling
7. Interface Design
8. Performance Considerations
9. References

1. Introduction

1.1 Scope of Document

This document describes the detailed internal design of SmartRec – a Context-Aware Hybrid Recommendation Framework for Smart City Infrastructure. It covers module-level architecture, algorithm definitions, data structures, function signatures, error-handling strategies, and interface specifications used during implementation.

1.2 Intended Audience

- Developers implementing or extending SmartRec modules
- Evaluators and academic reviewers
- QA engineers performing unit and integration testing
- Project reviewers and industry mentors

1.3 System Overview

SmartRec is a web-based, AI-powered recommendation system that provides personalized urban service recommendations to citizens and visitors of Smart Cities. The system accepts a user-selected infrastructure item and a contextual mode (Resident, Tourist, Emergency, or Night), then applies a multi-dimensional feature matrix — built from TF-IDF tags, One-Hot encoded categories, and normalized numeric scores — to compute Cosine Similarity. A hybrid scoring formula blends content similarity with global popularity to surface the most contextually appropriate city services. The entire pipeline is served through an interactive Streamlit dashboard with explainability and visual analytics components.

2. Detailed Module Design

2.1 Modules

1. Input & Context Module

- Accepts infrastructure item selection from dropdown
- Accepts user context mode: Resident, Tourist, Emergency, Night
- Validates user selections before passing to engine
- Triggers re-computation of contextual weights

2. Data Ingestion & Preprocessing Module

- Reads CSV/JSON city infrastructure datasets using Pandas
- Applies NLP semantic parsing on tags and accessibility_features columns
- Normalizes comma-separated tag strings into lowercase token lists
- Feature Engineering Pipeline:
 - TF-IDF Vectorization: Converts descriptive tags into a sparse TF-IDF matrix
 - One-Hot Encoding: Encodes location_zone and primary_category as binary vectors
 - Metric Scaling: Normalizes popularity_score and distance_km to [0, 1] range

3. Recommendation Engine (Core Logic Module)

- Builds combined feature matrix from TF-IDF, One-Hot, and scaled numeric columns
- Applies contextual weight vector based on selected mode
- Computes Cosine Similarity between selected item and all infrastructure entries
- Applies Hybrid Scoring: $\text{Hybrid Score} = (\text{Content Similarity} \times 0.8) + (\text{Global Popularity} \times 0.2)$
- Returns ranked list of top-N recommendations

4. Explainability Engine Module

- Performs set-intersection on tag lists of selected item and top recommendation
- Identifies shared category, zone, or feature attributes
- Generates human-readable justification strings
- Example output: "Shared 3 common features: mobility, electric-charging, sustainable"

5. Visualization Module

- Generates interactive Plotly heatmaps of the full similarity matrix
- Renders bar charts of hybrid scores for top recommendations
- Produces Matplotlib/Seaborn distribution plots for data exploration
- Saves static chart images for report export

6. Persistence & Caching Module

- Caches computed similarity matrices per context mode
- Serves cached results on repeated queries with zero latency
- Invalidates cache on dataset reload or context switch

7. Output & Dashboard Module

- Renders premium dark-mode Streamlit UI with custom CSS
- Displays top-N recommendations with metadata cards
- Shows KPI summary: total services, anomaly count, selected mode
- Provides downloadable report in CSV format

3. Algorithm Design

3.1 Hybrid Recommendation Algorithm

Steps:

1. Accept selected infrastructure item and user context mode from Input Module
2. Load and preprocess city infrastructure dataset (CSV/JSON) via Pandas
3. Apply NLP tokenization on tags and accessibility_features columns
4. Build TF-IDF matrix from tag features using TfidfVectorizer (Scikit-learn)
5. Apply One-Hot Encoding on categorical columns (location_zone, primary_category)
6. Normalize numeric columns (popularity_score, distance_km) using MinMaxScaler
7. Concatenate all feature matrices into a unified combined_matrix
8. Apply contextual weight vector based on selected mode (Emergency / Night / Tourist / Resident)
9. Compute Cosine Similarity between selected item vector and all other item vectors
10. Calculate Hybrid Score = (Content Similarity × 0.8) + (Global Popularity × 0.2)
11. Sort results by Hybrid Score and return top-N recommendations

3.2 Cosine Similarity Formula

$$\text{Cosine Similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Where A and B are feature vectors of two infrastructure items. A score of 1.0 indicates perfect semantic and functional alignment; 0.0 indicates no similarity.

3.3 Hybrid Scoring Formula

$$\text{Hybrid Score} = (\text{Content Similarity} \times 0.8) + (\text{Global Popularity} \times 0.2)$$

This formula ensures niche but highly relevant services are not buried by high-traffic locations.

3.4 Contextual Weight Profiles

Mode	Weight Focus	Boosted Features
Resident	Utility & Proximity	distance_km, civic category, utility tags
Tourist	Popularity & Leisure	popularity_score, leisure tags, landmarks
Emergency	Safety & Accessibility	medical tags, is_accessible, safety category
Night	24/7 Availability	is_24x7 flag, proximity, transit tags

4. Data Structures

Data Element	Structure / Type	Description
City Infrastructure Dataset	Pandas DataFrame	Primary dataset of all urban services and infrastructure items
Tag Tokens	List[str]	Normalized lowercase token list parsed from comma-separated tag strings
TF-IDF Matrix	scipy.sparse.csr_matrix	Sparse matrix of TF-IDF vectors for all infrastructure tag features
One-Hot Matrix	NumPy ndarray	Binary encoded matrix for location_zone and primary_category columns
Scaled Numeric Matrix	NumPy ndarray	MinMax-scaled array of popularity_score and distance_km
Combined Feature Matrix	NumPy ndarray (concatenated)	Unified matrix of TF-IDF, One-Hot, and numeric features
Contextual Weight Vector	Python Dictionary	Mode-keyed dictionary of feature weights per context profile
Cosine Similarity Matrix	NumPy ndarray (N×N)	Pairwise similarity scores across all infrastructure items
Recommendation Output	Pandas DataFrame	Ranked list of top-N items with scores and metadata
Similarity Cache	Python Dictionary (dict)	In-memory cache keyed by context mode storing computed matrices
Explanation Output	str	Human-readable justification string for the top recommendation

5. Function-Level Design

Function Name	Module	Input	Output	Description
load_data(filepath)	Data Ingestion	CSV/JSON path (str)	Pandas DataFrame	Loads and validates the city infrastructure dataset
parse_tags(tag_string)	Preprocessing	Comma-separated str	List[str]	Tokenizes, lowercases, and normalizes tag strings
build_tfidf_matrix(df)	Preprocessing	DataFrame	sparse csr_matrix	Applies TfidfVectorizer on the combined tags column
encode_categories(df)	Preprocessing	DataFrame	ndarray	Applies One-Hot Encoding on categorical columns
scale_numerics(df)	Preprocessing	DataFrame	ndarray	MinMax-scales numeric columns to [0,1] range
build_feature_matrix(df)	Engine	DataFrame	ndarray	Concatenates all feature matrices into a unified matrix
apply_context_weights(matrix, mode)	Engine	ndarray, str	ndarray	Multiplies feature matrix by contextual weight profile
compute_similarity(matrix)	Engine	ndarray	ndarray (N×N)	Computes pairwise Cosine Similarity across all items
get_recommendations(item, mode, n)	Engine	str, str, int	DataFrame	Returns top-N hybrid-scored

Function Name	Module	Input	Output	Description
				recommendations
compute_hybrid_score(sim, pop)	Engine	float, float	float	Calculates $(sim \times 0.8) + (pop \times 0.2)$ hybrid score
explain_recommendation(item_a, item_b)	Explainability	str, str	str	Returns set-intersection of tags as human-readable text
render_heatmap(sim_matrix)	Visualization	ndarray	Plotly Figure	Renders interactive infrastructure similarity heatmap
cache_matrix(mode, matrix)	Caching	str, ndarray	None	Stores computed matrix in the in-memory similarity cache
download_report(df)	Output	DataFrame	CSV file	Exports anomaly/recommendation results as downloadable CSV

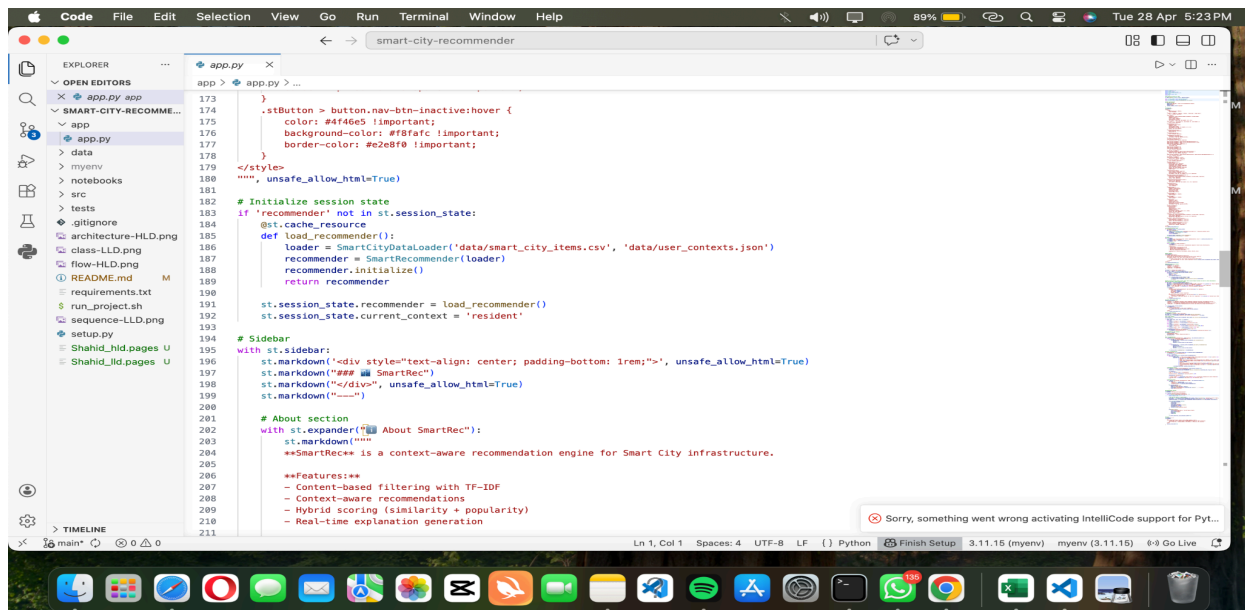
6. Error Handling

Error Condition	Handling Strategy	User Feedback
Invalid or missing file upload	Validate file presence and extension before processing	Display friendly error: 'Please upload a valid CSV/JSON file'
No numeric columns in dataset	Check dtypes after load; skip numeric scaling if absent	Notify user to upload a dataset with numeric features
Empty or corrupted dataset	Catch pandas read exceptions with try-except	Display: 'Dataset could not be loaded. Please check file format'
Selected item not found in dataset	Validate item name against DataFrame index before similarity lookup	Show: 'Selected service not found. Please choose from the list'
Division by zero in normalization	Add epsilon (1e-10) guard in MinMaxScaler to prevent zero-division	Processing continues silently; logged internally
No 24/7 services in Night Mode	Return available services sorted by proximity if is_24x7 filter yields empty results	Display: 'No 24/7 services found. Showing nearest alternatives'
ML model file not found	Check for model file on startup; fall back to Z-Score / statistical mode	Inform user: 'ML model unavailable. Using statistical similarity'
Similarity cache miss	Recompute matrix on cache miss and re-store result	Transparent to user; slight delay on first query per context
Visualization rendering failure	Wrap Plotly/Matplotlib calls in try-except; skip chart if error	Show: 'Chart could not be rendered for this dataset'

7. Interface Design

7.1 User Interface (Streamlit Dashboard)

- Premium dark-mode web dashboard with custom CSS styling
- Sidebar: Infrastructure item selector (dropdown) and context mode toggle (Resident / Tourist / Emergency / Night)
- Main Panel: KPI summary cards — Total Services, Top Match Score, Context Mode, Matched Features
- Recommendation Cards: Top-N results displayed with service name, category, zone, distance, popularity, and hybrid score
- Explainability Panel: Human-readable justification of top recommendation citing shared tags and attributes
- Visualization Tab: Interactive Plotly heatmap of full city infrastructure similarity matrix
- Report Download: One-click CSV export of current recommendation results



7.2 Internal Module Interfaces

Interface	From Module	To Module	Data Passed
load_data → build_feature_matrix	Data Ingestion	Recommendation Engine	Cleaned Pandas DataFrame
build_feature_matrix → compute_similarity	Preprocessing	Engine Core	Combined NumPy ndarray
compute_similarity → get_recommendations	Engine Core	Output Module	N×N Cosine Similarity ndarray

Interface	From Module	To Module	Data Passed
get_recommendations → explain_recommendation	Engine Core	Explainability Engine	Top item pair (str, str)
get_recommendations → render_heatmap	Engine Core	Visualization Module	Similarity Matrix ndarray
get_recommendations → download_report	Engine Core	Output Module	Ranked Results DataFrame

7.3 Output Interface

The dashboard displays the following outputs per recommendation query:

- Top-N recommendation list with hybrid scores and metadata
- Explainability text with shared feature justification
- Interactive similarity heatmap (Plotly)
- Downloadable CSV report of results

8. Performance Considerations

- In-memory processing via Pandas and NumPy — no database overhead required
- SciPy sparse matrix operations used for TF-IDF to minimize memory footprint
- Similarity Cache stores computed matrices per context mode — subsequent queries served at zero latency
- Selective feature scaling applied only to present numeric columns — avoids unnecessary computation
- TF-IDF vectorization is computed once at startup and reused across all queries
- Plotly heatmaps are rendered client-side in the browser for smooth interactivity on large matrices
- Streamlit caching decorators (`@st.cache_data`) prevent redundant data loading on UI interaction
- For datasets exceeding 10,000 items, sparse matrix representations keep memory usage under control
- Future scalability path: asynchronous task queues and multi-column parallel similarity computation

9. References

- Streamlit Documentation — <https://docs.streamlit.io>
- Pandas Documentation — <https://pandas.pydata.org/docs>
- NumPy Documentation — <https://numpy.org/doc>
- Scikit-learn Documentation — <https://scikit-learn.org/stable>
- SciPy Documentation — <https://docs.scipy.org>
- Plotly Documentation — <https://plotly.com/python>
- Matplotlib Documentation — <https://matplotlib.org>
- Isolation Forest — Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). IEEE International Conference on Data Mining
- TF-IDF Reference — Salton, G., & McGill, M. J. Introduction to Modern Information Retrieval
- Cosine Similarity — Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval